

# Security Audit

AZ Reserves Ltd (BaaS)

# Table of Contents

|                                   |    |
|-----------------------------------|----|
| Executive Summary                 | 4  |
| Project Context                   | 4  |
| Audit Scope                       | 7  |
| Security Rating                   | 8  |
| Intended Smart Contract Functions | 9  |
| Code Quality                      | 10 |
| Audit Resources                   | 10 |
| Dependencies                      | 10 |
| Severity Definitions              | 11 |
| Status Definitions                | 12 |
| Audit Findings                    | 13 |
| Centralisation                    | 26 |
| Conclusion                        | 27 |
| Our Methodology                   | 28 |
| Disclaimers                       | 30 |
| About Hashlock                    | 31 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

## Executive Summary

The Black Tie Asset Hub team partnered with Hashlock to conduct a security audit of their UserManager.sol, AuthorityManager.sol, Token.sol and other smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

## Project Context

The objective of the AZ Reserves project is to establish a transparent and independently verifiable reserve framework supporting gold and silver assets held for asset-backed financial and tokenisation activities. The project aims to ensure reserves are properly recorded, securely managed, and subject to independent audit and reporting processes that confirm the existence, allocation, and integrity of underlying assets. AZ Reserves seeks to provide institutional-grade governance and compliance standards to support financing, liquidity, and structured investment activities while maintaining strict asset segregation and accountability across all reserve holdings managed within the AZ Reserves framework.

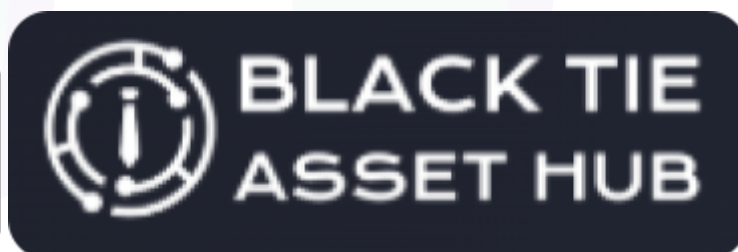
**Project Name:** AZ Reserves Ltd

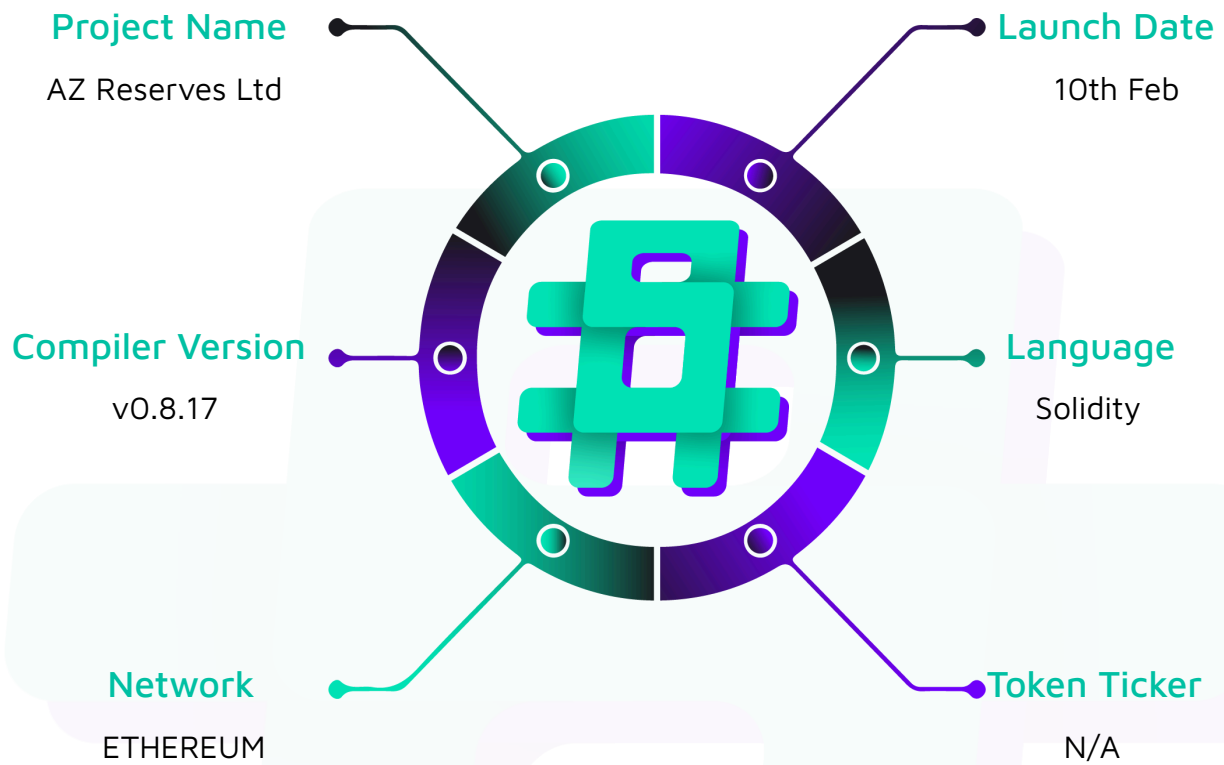
**Project Type:** BaaS

**Compiler Version:** 0.8.17

**Website:** <https://btassethub.digital/>

**Logo:**



**Visualised Context:**

**Project Visuals:**



**#hashlock.**

Hashlock Pty Ltd



## Audit Scope

We at Hashlock audited the solidity code within the AZ Reserves Ltd project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| Description                   | AZ Reserves Ltd Smart Contracts          |
|-------------------------------|------------------------------------------|
| Platform                      | Ethereum / Solidity                      |
| Audit Date                    | January, 2026                            |
| Contract 1                    | Factory.sol                              |
| Contract 2                    | RequestManager.sol                       |
| Contract 3                    | Token.sol                                |
| Audited GitHub Commit Hash    | 1ed3183f54d2b9e0d7a4863e733656d8008632c4 |
| Fix Review GitHub Commit Hash | 376a5a8eeedf43f8f3e04b9f7beb8795bdd3b6c9 |

## Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

### Hashlock found:

2 High severity vulnerabilities

4 Medium severity vulnerabilities

1 Low severity vulnerabilities

2 QA

**Caution:** Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.



## Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Actual Behaviour                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Token.sol</b> <ul style="list-style-type: none"> <li>- Allows the owner to:             <ul style="list-style-type: none"> <li>- Mint, Pause/Unpause transfers</li> <li>- Freeze/Unfreeze accounts</li> <li>- Force transfer</li> <li>- Update the factory address</li> </ul> </li> <li>- Allows users to:             <ul style="list-style-type: none"> <li>- Transfer, transferFrom or approve</li> <li>- Burn their own tokens</li> </ul> </li> </ul>                                                    | <b>Contract achieves this functionality.</b> |
| <b>Factory.sol</b> <ul style="list-style-type: none"> <li>- Allows the owner to:             <ul style="list-style-type: none"> <li>- Set the requestManager</li> <li>- Set global category percentages</li> <li>- Deploy new token clones</li> </ul> </li> <li>- Allows the RequestManager to:             <ul style="list-style-type: none"> <li>- Update per-token accounting</li> <li>- Distribute funds</li> <li>- Trigger category share distribution/mint</li> </ul> </li> </ul>                         | <b>Contract achieves this functionality.</b> |
| <b>RequestManager.sol</b> <ul style="list-style-type: none"> <li>- Allows the owner to:             <ul style="list-style-type: none"> <li>- Update factory, userManager and receiverAddress</li> </ul> </li> <li>- Allows token agent to:             <ul style="list-style-type: none"> <li>- Accept/Reject buy requests</li> </ul> </li> <li>- Allows users agent to:             <ul style="list-style-type: none"> <li>- Request tokens</li> <li>- Quote token amounts, read prices</li> </ul> </li> </ul> | <b>Contract achieves this functionality.</b> |

## Code Quality

This audit scope involves the smart contracts of the AZ Reserves Ltd project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the AZ Reserves Ltd project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

## Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description                                                                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| Medium       | Medium-level difficulties should be solved before deployment, but won't result in loss of funds.                                                                                                      |
| Low          | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                        |
| Gas          | Gas Optimisations, issues, and inefficiencies.                                                                                                                                                        |
| QA           | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.                            |

## Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance        | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolved</b>     | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.                                                                                   |
| <b>Acknowledged</b> | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| <b>Unresolved</b>   | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.                                                                                                                                                                                               |

# Audit Findings

## High

### [H-01] Token#forceTransfer - Frozen Token Accounting Is Not Enforced

#### Description

Frozen token mechanism is not enforced on standard transfers, and the current arithmetic in `forceTransfer()` can revert if `frozenTokens[from] > balanceOf(from)`, which is reachable due to missing caps on `frozenTokens`.

#### Vulnerability Details

The contract tracks `frozenTokens[user]`, but standard transfers only check `isFrozen` and do not restrict spending based on `frozenTokens`. That makes the “frozen amount” unenforced for normal user transfers.

Separately, `forceTransfer()` computes `balanceOf(from) - frozenTokens[from]`. If `frozenTokens[from]` exceeds the current balance, Solidity <sup>0.8</sup> underflow reverts, preventing forced recovery transfers. This state is reachable because `freezeTokens()` only checks the user’s balance at the time of each freeze and then adds to `frozenTokens` without maintaining a durable invariant against later balance changes (or multiple freezes over time).

#### Impact

Users can transfer tokens that are intended to be frozen, undermining the feature’s implied compliance purpose. Separately, if `frozenTokens[from]` ever exceeds the live balance, admin recovery via `forceTransfer` can become impossible due to underflow reverts, potentially leaving balances stuck in accounts that should be recoverable.

## Recommendation

Enforce frozen-token spendability in one place (`_update()`), for example by requiring `value <= balanceOf(from) - frozenTokens[from]` on transfers, and by explicitly defining the expected behavior for mint and burn paths.

Prevent `frozenTokens[user]` from exceeding the user's balance either by capping on freeze/unfreeze or designing it as escrow/lock that cannot become inconsistent.

## Status

Resolved

## [H-02] RequestManager#\_processRejection - Inconsistent Accounting In Rejection Causes Stuck Requests And Failed Refunds

### Description

The rejection path uses inconsistent units for `Factory.tokenInfo[token].locked` and attempts to pay the one time fee from the `RequestManager` contract despite custody being in the `Factory`, which can make rejections revert and block refunds.

### Vulnerability Details

On request creation,

`RequestManager.requestTokens()` increases `Factory.tokenInfo[token].locked` by `_tokenAmount` and transfers the full `_buyAmount` to the `Factory`.

On rejection,

`_processRejection()` calls `factory.updateTokenLocked(..., _requestInfo.buyAmount, false)`. This subtracts the *buy token amount* (USDC/USDT units) from a value that was increased in *token units*, and will revert if `locked < buyAmount`. Separately, `_processRejection()` refunds `_buyAmount + platformFee` via the `Factory`, but then attempts to pay `oneTimeFee` via `_buyToken.safeTransfer(receiverAddress, oneTimeFee)` from the `RequestManager` contract itself. Since funds were transferred to the `Factory` at request time, `RequestManager` typically has no balance and this transfer can revert whenever `oneTimeFee > 0`.

```
// requestTokens

factory.updateTokenLocked(_tokenAddress, _tokenAmount, true);

_buyToken.safeTransferFrom(msg.sender, address(factory), _buyAmount);

// _processRejection

factory.updateTokenLocked(_requestInfo.tokenAddress, _requestInfo.buyAmount, false);

factory.transferTokens(_buyToken, _user, _requestInfo.buyAmount +
_requestInfo.platformFee);
```



```
_buyToken.safeTransfer(receiverAddress, _requestInfo.oneTimeFee);
```

## Impact

Authorized agents may be unable to reject or refund requests. This can block refunds and leave requests stuck as pending. As a result, the locked value drifts over time and can no longer be safely used for accounting or downstream logic.

## Recommendation

Define the unit and intended meaning of locked and keep it consistent. If it is meant to represent outstanding token amounts, lock/unlock `requestInfo.tokenAmount` on request/reject/accept (not `buyAmount`). Ensure the one time fee payment is made from the same custody location as the buy funds (i.e. pay it via the Factory transfer function, or split the custody at request time so RequestManager actually holds the fee).

## Status

Resolved

## Medium

### [M-01] RequestManager#rejectBuyRequest - Missing !isCompleted Check Allows Contradictory States

#### Description

Requests have two independent flags (`isCompleted`, `isRejected`) but state transitions do not enforce mutual exclusion, allowing the same request to be both accepted and rejected.

#### Vulnerability Details

The acceptance path checks both `!isCompleted` and `!isRejected` before setting `isCompleted = true`. However, the rejection path only checks `!isRejected` and does not prevent rejecting an already completed request. This can still allow a request to end up both accepted and rejected, breaking settlement invariants.

```
// from _processNonPledgedRequest (accept path)

require(!_requestInfo.isCompleted, "Request already completed");
require(!_requestInfo.isRejected, "Request already rejected");

_requestInfo.isCompleted = true;

// from _validateRejectRequest (reject path)

require(!_requestInfo.isRejected, "Request already rejected");
```

#### Impact

`RequestManager` can enter contradictory states for the same request, breaking settlement invariants and accounting. In the worst case, an authorized token owner could accept (distribute tokens) and later reject (refund principal) the same request, creating a path to overpayment or loss.

## Recommendation

Replace two booleans with a single enum `RequestState { Pending, Accepted, Rejected }`, or require both flags to be `false` before either transition. Ensure the state change is one way.

## Status

Resolved

## **[M-02] RequestManager#acceptBuyRequest** - locked Accounting Never Decreases On Accept

### **Description**

`Factory.tokenInfo[token].locked` increases on request and decreases on rejection, but no decrease occurs on acceptance.

### **Vulnerability Details**

`locked` is increased when a user requests tokens, but it is never reduced on acceptance. As a result, `locked` only ever goes up for accepted requests and no longer reflects pending obligations unless you handle that meaning off-chain.

### **Impact**

`locked` becomes inflated over time, so it cannot be relied upon to represent outstanding refundable principal or pending obligations.

### **Recommendation**

Decrease `locked` when a request is accepted exactly once by subtracting the stored `tokenAmount` at acceptance time.

### **Status**

Resolved

## [M-03] Token#mint, \_mint - totalSupply Is Not Enforced As A Cap And Minting Is Uncapped

### Description

The Token contract allows the owner to mint arbitrary amounts without any cap, while `FactoryLib.TokenInfo.totalSupply` exists and is validated as `> 0`, which can be misinterpreted as an enforced maximum.

### Vulnerability Details

`Token.mint()` is `onlyOwner` and calls `_mint(to, amount)` without any cap. This might be an economic risk if it was meant to be capped.

```
function mint(address to, uint256 amount) public onlyOwner {  
    _mint(to, amount);  
    IFactory(factory).emitMintRequested(to, amount);  
}
```

### Impact

If buyers or integrators assume the offering has a hard max supply based on `FactoryLib.TokenInfo.totalSupply`, that assumption is false. The owner can inflate supply at any time, which can dilute holders and invalidate economic and security assumptions.

### Recommendation

If supply must be capped, store a `maxSupply` on initialization and enforce `totalSupply() + amount <= maxSupply` in `mint()`. If supply is intentionally uncapped, clearly document that minting is uncapped.

### Status

Acknowledged

## [M-04] RequestManager#acceptBuyRequest - Pledge Flow Bypasses Request Validation And Does Not Mark Requests Completed

### Description

When `_isPledge[i] == true`, `acceptBuyRequest()` skips the request completion logic and proceeds directly to token distribution, which can mint tokens without enforcing one way request state transitions.

### Vulnerability Details

Non-pledge path calls `_processNonPledgedRequest()`, which validates the request exists, checks that the passed `_tokenAmount` matches the stored request, checks `!isCompleted` and `!isRejected`, then marks the request `isCompleted = true` before transferring fees.

However, when `_isPledge[i]` is true, it skips `_processNonPledgedRequest()` and still calls `_processTokenDistribution()`, which mints/distributes tokens and emits an acceptance event. Meaning pledge acceptances are not forced to correspond to a stored request, and requests are not marked completed in this branch.

```
if (!_isPledge[i]) {  
    _processNonPledgedRequest(  
        _user,  
        _tokenAddress,  
        _tokenAmount,  
        _buyId  
    );  
}  
  
_processTokenDistribution(  
    _user,  
    _tokenAddress,  
    _tokenAmount,
```

```
_buyId,  
  
_investmentId  
);
```

### Impact

Pledge mints are not bound to stored request data and do not advance request state, which increases the risk of operational mistakes, double processing and inconsistent off-chain tracking.

### Recommendation

Make pledge processing use a coherent one way state machine as well (i.e. a single RequestState enum), or separate pledge minting into a dedicated function with explicit semantics that does not pretend to accept a request ID.

### Status

Acknowledged

## Low

### [L-01] RequestManager#requestTokens - tokenAmount Is User Supplied And Not Validated Against On-Chain Pricing

#### Description

`RequestManager.requestTokens()` accepts `_tokenAmount` as user input and stores it in the request without recomputing or validating it using on-chain pricing.

#### Vulnerability Details

`RequestManager` includes `calculateTokenAmount(...)` which uses the oracle and token metadata, but `requestTokens()` does not call it. Instead, it stores the user provided `_tokenAmount`, and the accept path later only checks that the agent passes the same `_tokenAmount` that was stored in the request.

This is not necessarily an exploit in a privileged agent flow since the agent can refuse to accept incorrect requests, but is an operational footgun.

#### Impact

A mistaken acceptance can mint too many or few tokens relative to the intended pricing.

#### Recommendation

Compute `_tokenAmount` on-chain in `requestTokens()` and store the computed value, or store the pricing inputs (amount, price, decimals) and validate them at acceptance time.

#### Status

Resolved



## QA

### [Q-01] RequestManager#requestTokens - buyType Is Not Validated

#### Description

Any `_buyType` value other than `0` is treated as USDT, which is an integration and UX footgun.

#### Recommendation

```
Add require(_buyType == 0 || _buyType == 1, "Invalid buy type"); in  
RequestManager.requestTokens().
```

#### Status

Acknowledged

## **[Q-02] Token#updateFactory** - `Token.updateFactory()` Can Brick Admin Actions Via Reverting Factory Calls

### **Description**

Admin functions call into `factory` for event emission. If the token owner sets `factory` to an address that reverts, those admin actions can revert as well.

### **Recommendation**

Validate `_factory` as a contract address (and optionally interface compatibility) in `updateFactory()`.

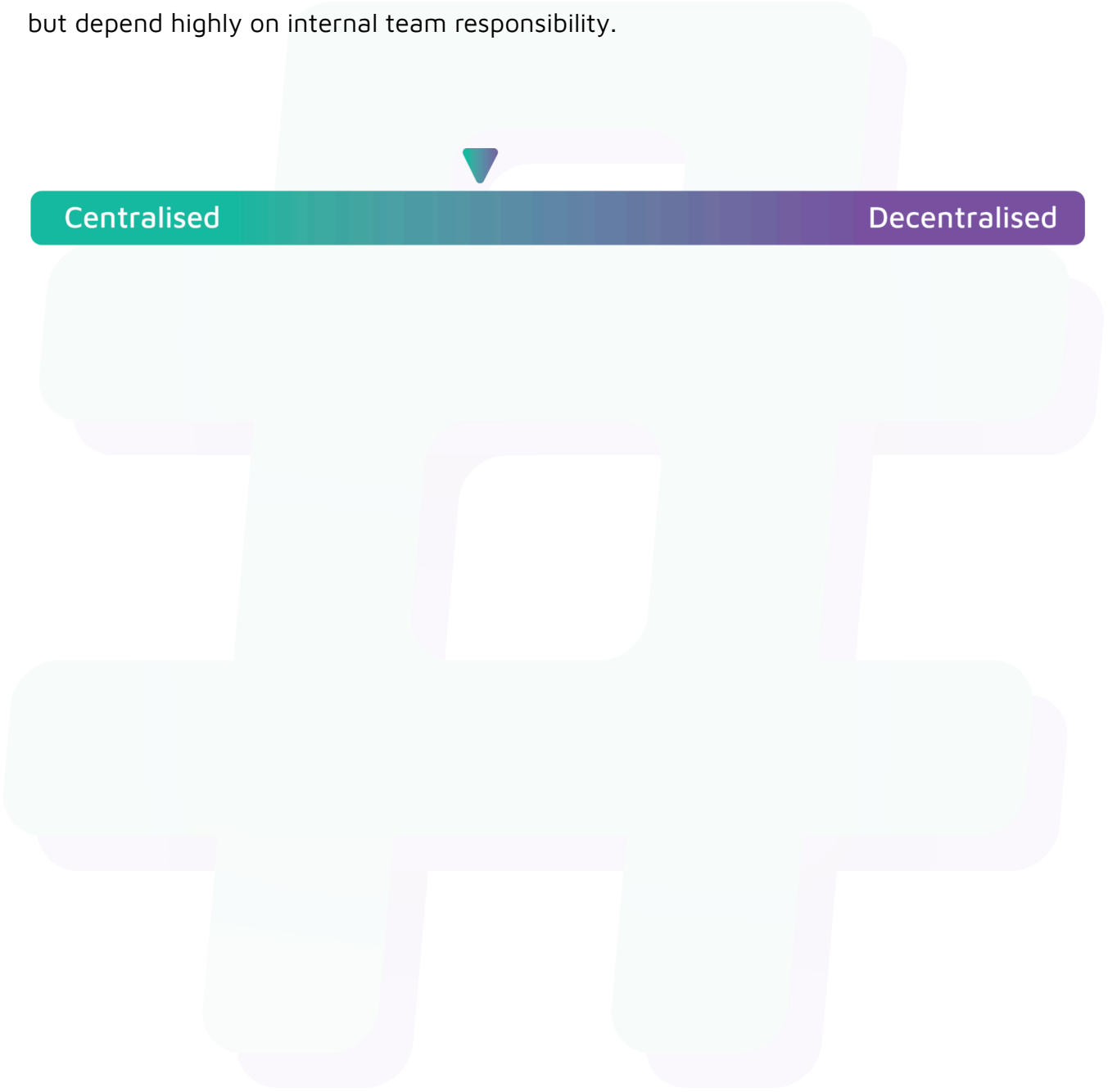
### **Status**

Resolved

## Centralisation

The AZ Reserves Ltd project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlock's analysis, the AZ Reserves Ltd project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

## Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

### **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

### **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)





**#hashlock.**